IN 213 **Solution**

Exercices

à faire et à rendre par courrier électronique

à `<francois • pessaux À ensta • fr` *avant* ~~*le 14 avril 2025, à 23h59s59*~~

À lire avant de vous lancer

Pour chacune des questions ci-dessous, il vous est demandé de fournir une fonction OCaml, ainsi que quelques tests **dans un commentaire**. Par exemple, l'énoncé de l'exercide 42 vous demandant d'écrire la fonction fact, calculant la factorielle d'un nombre, en vous indiquant qu'on attend une fonction de type (int → int), s'écrirait :

> 42. $\boxed{\mathsf{fact} : \mathsf{int} \to \mathsf{int}}$ Calculer la factorielle d'un entier.

et votre réponse pourrait être :

```
let rec fact n =
  if n ≤ 0 then 1 else n × fact (n − 1) ;;

(* Tests: fact(1) = 1; fact(0) = 1; fact(5) = 120; fact(−3) = 1 *)
```

La réponse à chacune des questions ci-dessous tient en quelques lignes. Ne perdez pas votre temps à chercher et recopier une réponse sur Internet : vous perdriez réellement votre temps (même si vous trouvez la réponse) car le but est que vous deveniez rapidement à l'aise avec OCaml pour les TDs à venir. Il y a l'air d'y avoir beaucoup d'exercices, mais ils sont simples à chaque fois. Ne soyez pas effrayés

Programmez en fonctionnel puisque le langage est conçu pour cela. Évitez au maximum les effets de bord. Dans les exercices ci-dessous, il n'y a pas besoin de boucle **for**, **while** ou de références par exemple. La récursion est votre amie dans ce cours...

Afin de me faciliter la tâche de vérification, merci de **nommer vos fonctions, vos types comme demandé** dans les questions.

1 Exercices sur les fonctions simples

1. $\boxed{\mathsf{id} : \alpha \to \alpha}$ Écrire la fonction identité.
2. $\boxed{\mathsf{cpt_voy} : \mathsf{string} \to \mathsf{int}}$ Retourner le nombre de voyelles dans une chaîne de caractères. Calculer la longueur d'une chaîne se fait par la fonction String.length. L'accès au caractère d'indice i dans une chaîne s se fait par s.[i]. Vous n'êtes pas obligé de considérer les accentuées.
3. $\boxed{\mathsf{somme_inter} : (\mathsf{float} \times \mathsf{float}) \to (\mathsf{float} \times \mathsf{float}) \to (\mathsf{float} \times \mathsf{float})}$ Retourner la "somme" de deux intervalles. Un intervalle est représenté par un couple de flottants dont la première composante est la borne inférieure et la seconde est la borne supérieure (on suppose que cette hypothèse est vérifiée). La "somme" est l'intervalle ayant pour min l'addition des mins entre eux et pour max l'addition des maxs entre eux.

4. $\boxed{\text{puiss_2} : (\alpha \to \alpha) \to \alpha \to \alpha}$ Écrire la fonction puiss_2 telle que puiss_2 f x calcule $f^2(x)$ (l'application de f 2 fois).
5. $\boxed{\text{base8} : \text{int} \to \text{string}}$ Retourner la chaîne de caractères représentant l'écriture en base 8 d'un entier passé en argument. La concaténation de chaînes se fait par l'opérateur infixe ^.

2 Exercices sur les types somme

1. $\boxed{\text{nom_feu} : \text{feu} \to \text{string}}$ Définir le type des feux tricolores (rouge / orange / vert) et une fonction retournant la chaîne de caractères correspondant à la valeur de feu reçue en argument.
2. $\boxed{\text{prof_max} : \alpha\ \text{arbre} \to \text{int}}$ Définir le type α arbre permettant de représenter des arbres binaires et écrire la fonction retournant la profondeur d'un arbre. On considère qu'une feuille a une hauteur de 1 (et non de 0) puisque l'on peut y stocker de l'information.

3 Exercices sur les listes

1. $\boxed{\text{dernier} : \alpha\ \text{list} \to \alpha}$ Trouver le dernier élément d'une liste, échouer (raise (Failure "trop courte")) si la liste est trop courte.
2. $\boxed{\text{avant_dernier} : \alpha\ \text{list} \to \alpha}$ Trouver l'avant-dernier élément d'une liste, échouer si la liste est trop courte.
3. $\boxed{\text{nieme} : \alpha\ \text{list} \to \text{int} \to \alpha}$ Trouver l'élément à la position n d'une liste, le **premier élément est à la position 0**, échouer si la liste est trop courte ou si $n < 0$.
4. $\boxed{\text{nieme_bis} : \alpha\ \text{list} \to \text{int} \to \alpha\ \text{option}}$ Trouver l'élément à la position n d'une liste (le premier élément est à la position 0), et renvoyer None lorsque la liste est trop courte ou si $n < 0$.
5. $\boxed{\text{copie} : \alpha\ \text{list} \to \alpha\ \text{list}}$ Copier une liste (copie dite «superficielle» : on ne copie que la structure de liste).
6. $\boxed{\text{renverse} : \alpha\ \text{list} \to \alpha\ \text{list}}$ Renverser une liste (produire une copie, à l'envers, de la liste initiale).
7. $\boxed{\text{sans_repetitions} : \alpha\ \text{list} \to \alpha\ \text{list}}$ Produire la copie d'une liste, où les répétitions d'éléments consécutifs ont été supprimées (on peut tester l'égalité entre deux éléments x et y en écrivant simplement x = y).
8. $\boxed{\text{trie_selon_la_longueur} : \alpha\ \text{list list} \to \alpha\ \text{list list}}$ Trier une liste de listes selon la longueur de ses éléments (utiliser la fonction List.sort de la bibliothèque standard d'OCaml).
9. $\boxed{\text{appl_1_sur_2} : (\alpha \to \beta) \to \alpha\ \text{list} \to \beta\ \text{list}}$ Appliquer une fonction reçue en argument à 1 élément sur 2 d'une liste (en commençant par le premier élément de la liste) et en oubliant ceux sur lesquels on n'applique pas la fonction reçue en argument.
10. $\boxed{\text{tableau_de_liste} : \alpha\ \text{array} \to \alpha\ \text{list}}$ Transformer un tableau en liste. La taille d'un tableau s'obtient en utilisant la fonction Array.length. Pour accéder à l'élément i d'un tableau t, la syntaxe est t.(i). Pour définir un tableau, on utilise la syntaxe (exemple) [|41 ; 4 ; 13 ; 64 ; 7|].
11. $\boxed{\text{separe} : \text{int list} \to (\text{int list}) \times (\text{int list})}$ Séparer une liste d'entiers en 2 listes, celle des entiers pairs et celle des entiers impairs. L'ordre des éléments dans chaque liste correspond à leur ordre dans la liste initiale. Par exemple, split [0 ;1 ;2 ;3 ;4 ;5] devra donner ([0 ; 2 ; 4], [1 ; 3 ; 5]).

4 Divers

On souhaite représenter des ensembles finis d'entiers et vérifier s'il existe une intersection non vide entre 2 ensembles. Un tel ensemble est donné soit par :

— `Interval of (int * int)` : un intervalle dont les bornes sont considérées incluses dans l'ensemble représenté,
— `List of int list` : une liste d'entiers.

Dans la représentation par un intervalle, par hypothèse le premier entier est la borne inférieure et le second la borne supérieure. Dans la représentation par une liste, les entiers n'ont pas à être ordonnés d'une manière particulière.

1. Écrivez la définition de type ensemble correspondant à la représentation d'un ensemble.
2. Écrivez la fonction intersec : ensemble → ensemble → bool qui prend 2 ensembles en argument et retourne le booléen disant si leur intersection est non vide.

Fin des exercices